

High-Speed 16-Radix Polynomial Multiplication on ARM Cortex-M4 with Recursive Karatsuba Layers

Minjoo Sim^{1[0000-0001-5242-214X]}, Hyunjun Kim^{1[0000-0001-6757-6109]}, Minwoo Lee^{1[0000-0002-2356-3055]}, and Hwajeong Seo^{1[0000-0002-2356-3055]}

Hansung University, Seoul 02876, Republic of Korea
{minjoos9797, khj930704, minunejip, hwajeong840}@gmail.com

Abstract. Polynomial multiplication over $\mathbb{F}_2[x]$ is a fundamental building block in code-based and lattice-based cryptography, particularly on lightweight embedded devices where dedicated carry-less multiply instructions are unavailable. This paper presents a high-speed, constant-time implementation of radix-16 polynomial multiplication on the ARM Cortex-M4, combining zero-padding with recursive Karatsuba layers. Building on the radix-16 decomposition proposed by Chen et al. in TCHES’21, we replace the conventional schoolbook inner multiplier with a multi-level Karatsuba scheme. This optimization reduces cycle counts on the ARM Cortex-M4 while preserving constant-time execution. To further optimize efficiency, the design minimizes packing and unpacking overhead by operating at 128-bit granularity and employs a five-stage pipeline—Decomposition, Padding, Multiplication, Unpadding, and Reassembly—implemented entirely with data-independent shifts, XORs, and masks. Experimental results on the Cortex-M4 show that our optimized `ct_poly32_mul_64_bit` implementation achieves up to 31% improvement over the best existing constant-time baseline, demonstrating the efficiency and scalability of recursive Karatsuba for resource-constrained cryptographic applications.

Keywords: Polynomial Multiplication · Software Implementation · Recursive Karatsuba · Cortex-M4

1 Introduction

Polynomial multiplication in $\mathbb{F}_2[x]$ is a core operation in many cryptographic primitives, including code-based and lattice-based post-quantum schemes. Its efficiency directly influences the practicality of encryption, decryption, key generation, and signature verification in resource-constrained environments. On high-performance CPUs, dedicated carry-less multiply instructions (e.g., Intel CLMUL) allow efficient realization of this task, often achieving single-cycle throughput for small field multiplications [1]. However, embedded devices such as the ARM Cortex-M4 lack such instructions, requiring alternative approaches

to achieve competitive performance without compromising security. This limitation poses a significant challenge since microcontrollers remain a dominant platform for secure IoT devices, medical sensors, and industrial control systems, where post-quantum cryptographic primitives are expected to be deployed in the near future [2,3,4].

The schoolbook method, also known as long multiplication, provides a simple and direct implementation by computing all pairwise coefficient products [5]. While conceptually straightforward, its quadratic complexity $O(n^2)$ leads to rapidly increasing cycle counts as operand sizes grow, which is prohibitive for many cryptographic applications. In contrast, Karatsuba’s divide-and-conquer approach reduces the complexity to $O(n^{\log_2 3}) \approx O(n^{1.585})$, offering tangible cycle savings for moderate operand sizes that frequently appear in code-based cryptosystems and structured lattice schemes [6,7].

On the ARM Cortex-M4 in particular, several works have investigated optimized polynomial multipliers. Erdem [8] explored comb and Karatsuba techniques for efficient software multiplication over $\mathbb{F}_2[x]$ on embedded platforms, while Kannwischer [9] evaluated multiplication strategies including Karatsuba and Toom–Cook in the context of NIST PQC candidates. Chen et al. [10] introduced a radix-16 packing scheme for Classic McEliece, demonstrating the benefits of bit-sliced convolution. More recent studies have further optimized lattice- and code-based schemes on this platform, such as ESPM-D for Dilithium [11] and HQC implementations leveraging Karatsuba multipliers [12].

These results highlight that packing and bit-sliced representations are effective in exploiting the native word size of 32-bit processors. By representing polynomials in packed formats, carry-less multiplication can be emulated with ordinary integer multiplication followed by masking and shifting. Nevertheless, most prior work continues to rely on schoolbook multiplication at the core, which leaves room for more efficient constant-time engines.

To address this gap, we explore recursive Karatsuba layers as a promising alternative. By reducing the number of base multiplications without introducing data-dependent control flow, and by combining them with constant-time masking, shifting, and packing routines, our approach provides both security and performance advantages. Building on these insights, this paper proposes a new design that integrates radix-16 zero-padding with recursive Karatsuba multipliers, tailored for the ARM Cortex-M4 platform.

The remainder of this paper is structured as follows. Section 2 reviews the Schoolbook and Karatsuba algorithms together with related work on polynomial multiplication. Section 3 presents our proposed method, Section 4 reports the performance evaluation, and Section 5 concludes with a discussion of future work.

Contributions

This paper makes the following contributions:

- **Recursive Karatsuba optimization:** We propose a constant-time multiplier that extends the radix-16 zero-padding framework of Chen et al. [10] by

introducing recursive multi-level Karatsuba layers in place of the traditional schoolbook multiplier.

- **Pipeline refinement:** We design a five-stage multiplication pipeline (Decomposition, Padding, Multiplication, Unpadding, Reassembly) that is implemented entirely with data-independent shifts, XORs, and masks. By operating at 128-bit granularity, the design minimizes packing and unpacking overhead on 32-bit processors.
- **Scalable implementations (64- and 128-bit variants):** Our implementation includes two constant-time designs: `ct_poly32_mul_64_bit` for efficiency and `ct_poly32_mul_128_bit` for scalability to larger operands. The 64-bit variant offers the best efficiency (up to 31% cycle-count reduction over the constant-time baseline [13]), while the 128-bit variant demonstrates the scalability of our approach to wider operands and larger field sizes. We further benchmark multiple recursion depths (L2, L3, L4) on the Cortex-M4, confirming the practical efficiency of recursive Karatsuba on resource-constrained devices.

2 Background

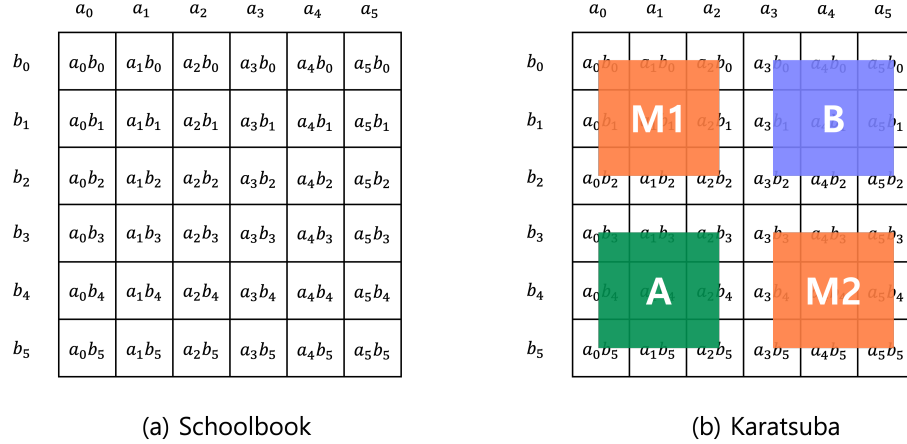


Fig. 1: (a) Schoolbook (direct) polynomial multiplication: each cell is a partial product $a_i b_j$; each output coefficient c_k is the sum along the diagonal $i + j = k$. (b) Karatsuba partition of the same operand: split inputs into lower/upper halves X_0, X_1 and compute A, B, M (highlighted). Recompose via Karatsuba to reduce base multiplications.

2.1 Schoolbook Multiplication

The *Schoolbook multiplication*, also called *long multiplication* or the *grade-school method*, follows the direct application of the distributive law. Given two n -digit numbers (or degree- $(n-1)$ polynomials)

$$x = \sum_{i=0}^{n-1} a_i B^i, \quad y = \sum_{j=0}^{n-1} b_j B^j,$$

Here, B denotes the radix used for representing polynomial coefficients (typically $B = 2$). the product expands as

$$x \cdot y = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} a_i b_j B^{i+j}.$$

This requires computing all n^2 pairwise products, leading to quadratic time complexity $O(n^2)$ [5]. Its strength lies in simplicity and straightforward implementation, as it essentially mirrors the manual multiplication process taught in elementary arithmetic.

Figure 1(a) illustrates this method: each cell corresponds to a partial product $a_i b_j$, and an output coefficient c_k is obtained by summing (XOR-ing in $\mathbb{F}_2$) all partial products on the diagonal $i + j = k$,

$$c_k = \sum_{i=0}^k a_i b_{k-i} \quad (\text{addition is XOR in } \mathbb{F}_2).$$

This representation maps naturally to word-oriented implementations: operands are split into fixed-size limbs (words or nibbles), partial products are formed by word multiplications, and the diagonal contributions are combined by shifts and XORs. On platforms without a dedicated carry-less multiply instruction (e.g., ARM Cortex-M4), the schoolbook approach can be realized by packing limbs into bit-spaced (radix) words and using native integer multiply operations together with shifts and XORs to emulate carry-less polynomial multiplication.

2.2 Karatsuba Multiplication

The *Karatsuba algorithm*, introduced by Karatsuba and Ofman in 1963 [6], improves upon the Schoolbook method using a divide-and-conquer strategy. This process is visualized in Figure 1(b).

First, the inputs are split into lower and upper halves:

$$x = x_1 K^m + x_0, \quad y = y_1 K^m + y_0.$$

In this context, K represents the splitting base (e.g., $K = 2^m$), which determines the partitioning width in the recursive formulation.

While a naive expansion of this form requires four sub-multiplications, Karatsuba’s key insight is that the result can be computed with only three:

$$\begin{aligned} z_2 &= x_1 y_1 \\ z_0 &= x_0 y_0 \\ z_1 &= (x_1 + x_0)(y_1 + y_0) - z_2 - z_0. \end{aligned}$$

The final product is then reconstructed by shifting and adding these partial products:

$$x \cdot y = z_2 K^{2m} + z_1 K^m + z_0.$$

This recursive approach reduces the time complexity to $O(n^{\log_2 3}) \approx O(n^{1.585})$ [14], making it the first widely known multiplication algorithm with sub-quadratic complexity.

2.3 32-bit Cortex-M4

The ARM Cortex-M4 is a widely deployed 32-bit microcontroller core that combines energy efficiency with moderate computational capability, making it a representative platform for cryptographic benchmarking [15,16]. It implements the ARMv7E-M instruction set, including Thumb-2 instructions, DSP extensions, and a hardware floating-point unit (FPU). With features such as a three-stage pipeline and dedicated multiplication-accumulation instructions (e.g., UMAAL), the Cortex-M4 provides efficient support for arithmetic-intensive applications while remaining suitable for resource-constrained embedded devices.

Due to these characteristics, the Cortex-M4 has become a standard platform for benchmarking lightweight and post-quantum cryptography. For instance, the `pqm4` framework provides a unified benchmarking environment for NIST PQC candidates on Cortex-M4 [17], and numerous studies have reported implementations of ECC, DSA, and lattice-based schemes on this architecture [18,19,11,20]. These works demonstrate that the Cortex-M4 is an appropriate and widely recognized target for evaluating high-performance cryptographic primitives in constrained environments.

2.4 Related Work on Polynomial Multiplication

LD multiplication [21] Lookup-table (LUT) based approaches such as the Lopez–Dahab (LD) method trade bitwise operations for table lookups and XORs, yielding fast word-oriented polynomial multiplication but exposing potential timing side channels unless implemented in constant time.

D. J. Bernstein’s 64-bit bit-sliced multiplier [13] D. J. Bernstein’s 64-bit bit-sliced carry-less multiplier removes data-dependent table lookups, but because it is implemented around 64-bit slices it incurs packing/unpacking overhead when executed on 32-bit targets (e.g., ARM Cortex-M4), which can reduce its practical benefit on such platforms.

Classic McEliece on the ARM Cortex-M4 (TCHES’21) [10] Chen *et al.* adopt a radix-16 (4-bit digit) bit-sliced packing to map small carry-less polynomial multiplications onto integer multiplications on 32-bit processors. A polynomial

$$a(x) = a_0 + a_1x + a_2x^2 + \cdots + a_7x^7 \in \mathbb{F}_2[x]$$

is represented as the 32-bit integer

$$A = a_0 \cdot 2^0 + a_1 \cdot 2^4 + a_2 \cdot 2^8 + \cdots + a_7 \cdot 2^{28},$$

and similarly for B . The integer product

$$A \cdot B = \sum_{i=0}^7 \sum_{j=0}^7 a_i b_j 2^{4(i+j)}$$

places the contribution $a_i b_j$ at bit position $4(i+j)$; consequently, the bit at index $4k$ of the integer product equals the carry-less coefficient c_k of the polynomial product. Thus, small carry-less convolutions can be realized by a sequence of 32-bit integer multiply-accumulate instructions (e.g. **UMLAL**) together with masking and shifts to extract and rearrange the relevant bits.

Because the Cortex-M4 lacks a dedicated **CLMUL** instruction, the authors implement the core multiplication loop in assembly using a few **UMLAL** (and when needed **MLA**) instructions for the polynomial phase and similarly for the reduction phase. By carefully designing the packing, masks/shifts and the assembly inner loop, the radix-16 scheme attains low cycle counts on 32-bit ARM cores at the cost of increased packing/unpacking and assembly complexity. The approach is therefore particularly effective when many small parallel multiplications are executed or when hand-tuned assembly is acceptable to amortize the packing overhead.

3 Proposed Method

Building on Chen et al.’s 16-radix decomposition, data layout, and zero-padding scheme [10], we retain their high-level structure and platform optimizations but replace the schoolbook inner multiplier with a recursive multiple Karatsuba engine. Packing/unpacking is performed once at 128-bit granularity rather than repeatedly at smaller widths, substantially reducing packing overhead on 32-bit targets and improving throughput and scalability for larger field sizes.

3.1 Zero-Padding Method

For a binary polynomial represented as

$$a = \sum_{i \geq 0} a_i 2^i \quad (a_i \in \{0, 1\}),$$

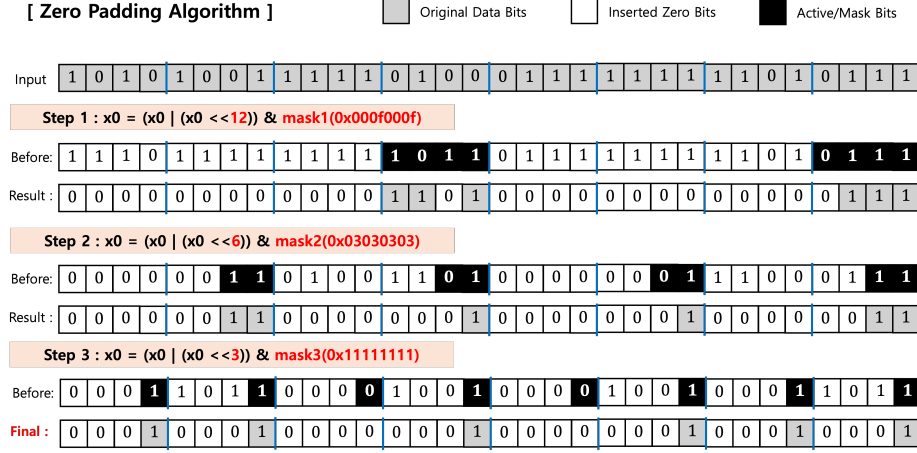


Fig. 2: Step-by-step execution of the zero-padding transformation on the 16-bit input $x_0 = 0xB4D2$. The process applies OR-shift-mask operations with three masks (`mask1`, `mask2`, and `mask3`) to insert zero bits between every original coefficient, yielding the 32-bit output $x_0 = 0x01000010$. Gray boxes indicate original bits, white boxes show inserted zeros, and black boxes mark active mask bits.

the zero-padding transformation is defined as

$$\text{Pad}(a) := \sum_{i \geq 0} a_i 2^{4i},$$

which inserts three zero bits between consecutive coefficients to prevent carry propagation during native integer multiplication. On 32-bit processors, this mapping is efficiently realized through a sequence of *shift-OR-mask* operations (e.g., shifts by 12, 6, and 3 positions).

The inverse transformation is defined as

$$\text{InvPad}(b) := \sum_{i \geq 0} \left(\left\lfloor \frac{b}{2^{4i}} \right\rfloor \bmod 2 \right) 2^i,$$

where $b = \text{Pad}(a)$. In other words, every fourth bit of b is extracted to recover the original word a , and $\text{InvPad}(\text{Pad}(a)) = a$ holds.

As illustrated in Figure 2, the `zero_padding` and `inv_zero_padding` macros implement these `Pad` and `InvPad` transformations. In Algorithm 2, Lines 3–6 correspond to the Padding stage, while Lines 10–11 correspond to the Unpadding stage.

3.2 Recursive Karatsuba Layers

Although Algorithm 1 presents the recursive structure in an abstract form, the base-case MUL requires an efficient, constant-time instantiation. Rather than

relying on a traditional schoolbook multiplication, our design realizes this base case through a recursive combination of Karatsuba multiplications, as shown in Figure 3.

Algorithm 1 Recursive Karatsuba on Zero-Padded Blocks

Require: Arrays X, Y of length n (zero-padded 32-bit words), where n is a power of two

Ensure: Array C of length $2n$, where each entry is either a padded 64-bit word (for `ct_poly32_mul_64_bit`) or a padded 128-bit block (for `ct_poly32_mul_128_bit`)

```

1: if  $n = 1$  then                                ▷ Base case: one 64-bit multiplication (2-word multiply)
2:    $C \leftarrow \text{MUL}(X_0, Y_0)$ 
3:   return  $C$ 
4: else
5:    $X_L \leftarrow (X_0, \dots, X_{n/2-1}), \quad X_H \leftarrow (X_{n/2}, \dots, X_{n-1})$ 
6:    $Y_L \leftarrow (Y_0, \dots, Y_{n/2-1}), \quad Y_H \leftarrow (Y_{n/2}, \dots, Y_{n-1})$ 
7:    $C_L \leftarrow \text{KARATSUBA}(X_L, Y_L, n/2)$ 
8:    $C_H \leftarrow \text{KARATSUBA}(X_H, Y_H, n/2)$ 
9:    $C_M \leftarrow \text{KARATSUBA}(X_L \oplus X_H, Y_L \oplus Y_H, n/2)$ 
10:   $T \leftarrow C_M \oplus C_L \oplus C_H$ 
11:  return  $C \leftarrow (C_H \ll n) \oplus (T \ll (n/2)) \oplus C_L$ 
12: end if

```

For comparison, the conventional schoolbook expansion for two 32-bit words is

$$XY = x_0y_0 + (x_0y_1 \oplus x_1y_0)2^{32} + x_1y_12^{64},$$

where $\oplus$ denotes addition over $\mathbb{F}_2$, which corresponds to the bitwise XOR operation between two 32-bit words in the implementation. This contrasts with our recursive Karatsuba-based instantiation, motivating the use of multiple Karatsuba multiplications in the constant-time implementation. The bitsliced Karatsuba multiplier further achieves high cycle efficiency per multiplication, making it well suited for resource-constrained embedded platforms.

The recursive Karatsuba structure of Algorithm 1 provides the multiplication core invoked in Algorithm 2 (Line 7), which is explicitly marked as the Multiplication stage in the overall pipeline of Figure 4.

3.3 Top-Level Multiplication Pipeline

The overall multiplication procedure is organized as an *algorithmic* five-stage pipeline (Decomposition, Padding, Multiplication, Unpadding, and Reassembly), as illustrated in Algorithm 2 and Figure 4. The term *pipeline* refers to the logical dataflow within our algorithm, distinct from the hardware instruction pipeline of the Cortex-M4 processor. This structured decomposition improves modularity and ensures that each stage can be implemented with data-independent operations.


```

1 // Constant-time 2-word Karatsuba multiplier (base case)
2 void ct_poly_karatsuba_mul_2word(uint64_t *c_out,
3                                 uint32_t *x_in,
4                                 uint32_t *y_in) {
5     uint32_t *x_part = x_in, *y_part = y_in;
6     uint32_t x_mid, y_mid;
7     uint64_t a_part, b_part, c_part;
8
9     #ifdef KARATSUBA_L4
10        // Karatsuba (L4)
11        a_part = ((uint64_t)x_part[0] * (uint64_t)y_part[0]) &
12            mask3_64; //  $x_0*y_0$ 
13        b_part = ((uint64_t)x_part[1] * (uint64_t)y_part[1]) &
14            mask3_64; //  $x_1*y_1$ 
15        x_mid = x_part[0] ^ x_part[1];
16        y_mid = y_part[0] ^ y_part[1];
17        c_part = ((uint64_t)x_mid * (uint64_t)y_mid) & mask3_64;
18            //  $(x_0 \oplus x_1) * (y_0 \oplus y_1)$ 
19        c_part ^= a_part ^ b_part;
20            // middle term
21
22        a_part ^= (c_part << 32);
23        b_part ^= (c_part >> 32);
24    #else
25        // Schoolbook fallback (carry-less)
26        a_part = ((uint64_t)x_part[0] * (uint64_t)y_part[0]) &
27            mask3_64; //  $x_0*y_0$ 
28        b_part = (((uint64_t)x_part[0] * (uint64_t)y_part[1]) &
29            mask3_64) ^
30            (((uint64_t)x_part[1] * (uint64_t)y_part[0]) &
31            mask3_64); //  $x_0*y_1 \oplus x_1*y_0$ 
32        a_part ^= (b_part << 32);
33            // add middle
34            at  $2^{32}$ 
35        b_part = (b_part >> 32) ^
36            (((uint64_t)x_part[1] * (uint64_t)y_part[1]) &
37            mask3_64); // add  $x_1*y_1$  at  $2^{64}$ 
38    #endif
39
40    // Result (padded 128-bit)
41    c_out[0] = a_part;
42    c_out[1] = b_part;
43 }

```

Fig. 3: Base constant-time two-word Karatsuba multiplier used at the lowest recursion layer.

In Algorithm 2, Lines 1–2 normalize the input length and allocate arrays to hold the zero-padded chunks. Lines 3–6 decompose the 32- or 128-bit input into 8-bit slices, each representing eight $\mathbb{F}_2$ coefficients in 16-radix bit-sliced form. These slices are then expanded via the Pad operation (Line 5), which inserts zeros between coefficient bits to yield zero-padded 32-bit words. On these padded inputs, recursive Karatsuba multiplication is applied (Line 7, Algorithm 1), producing arrays of padded 64-bit partial products. The results are subsequently compressed by Unpad (Line 10), which extracts every fourth bit to obtain compact 16-bit words, and are stored in array Z (Line 11). Finally, in Lines 13–16, pairs of 16-bit words are reassembled into 32-bit outputs using $c_out[i] = low \mid (high \ll 16)$, and in Line 17 the output words are placed at byte-aligned offsets to form the final product.

As summarized in Figure 4, the five stages—Decomposition $\rightarrow$ Padding $\rightarrow$ Multiplication $\rightarrow$ Unpadding $\rightarrow$ Reassembly—are implemented with data-independent shifts, XORs, and masks to ensure constant-time execution on the Cortex-M4. Algorithm 2 explicitly reflects this pipeline: Lines 3–6 correspond to Decomposition and Padding (Pad is applied at Line 5); Line 7 invokes Algorithm 1 as the Multiplication core; Lines 10–11 implement Unpadding; Lines 13–16 perform the 16-to-32-bit Reassembly; and Line 17 assembles the final output at byte-aligned offsets.

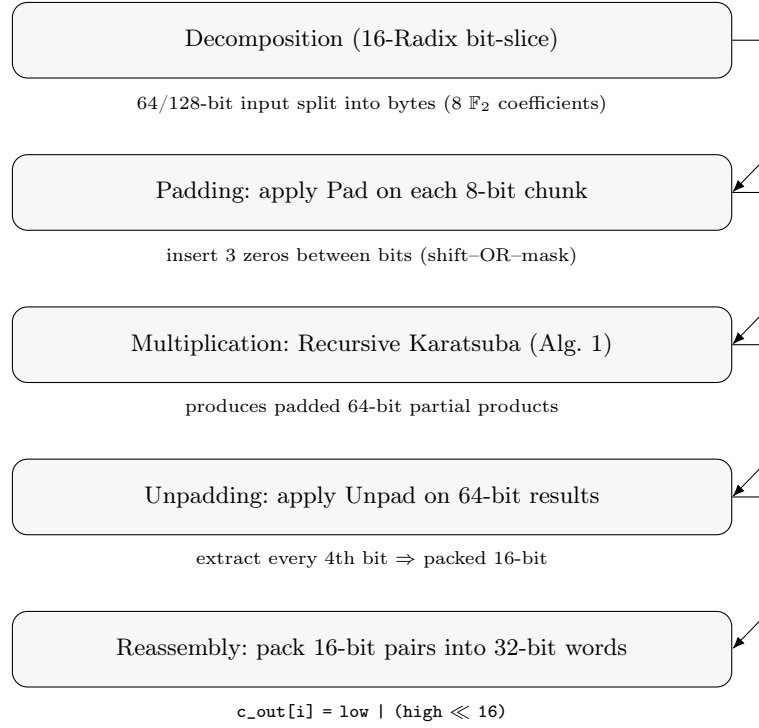
Algorithm 2 Top-level constant-time multiplication pipeline, illustrated for the 64-bit base multiplier; the 128-bit variant applies the similar structure with wider Unpad and Reassembly stages.

Require: Bitstrings A, B of length N (multiple of 8)
Ensure: Product bitstring P (length $\leq 2N$)

```

1:  $k \leftarrow N/8$  ▷ number of 8-bit chunks
2: Allocate arrays  $X[0..k-1], Y[0..k-1]$  ▷ zero-padded 32-bit words
3: for  $i \leftarrow 0$  to  $k-1$  do
4:    $a_i \leftarrow i$ -th 8-bit chunk of  $A$ ;  $b_i \leftarrow i$ -th 8-bit chunk of  $B$ 
5:    $X[i] \leftarrow \text{Pad}(a_i)$ ;  $Y[i] \leftarrow \text{Pad}(b_i)$ 
6: end for
7:  $C \leftarrow \text{KARATSUBA}(X, Y, k)$  ▷ Algorithm 1
8: Allocate array  $Z[0..(4k-1)]$  ▷ packed 16-bit words
9: for  $j \leftarrow 0$  to  $2k-1$  do
10:   $(\ell, h) \leftarrow \text{Unpad}(C[j])$ 
11:   $Z[2j] \leftarrow \ell$ ;  $Z[2j+1] \leftarrow h$ 
12: end for
13: Allocate array  $P_w[0..(2k-1)]$  ▷ final 32-bit words
14: for  $t \leftarrow 0$  to  $2k-1$  do
15:   $P_w[t] \leftarrow Z[2t] \mid (Z[2t+1] \ll 16)$ 
16: end for
17: Assemble  $P$  from  $P_w$  at byte-aligned offsets
18: return  $P$ 

```



Example bit patterns:



Fig. 4: Five-stage dataflow of the proposed multiplier, shown for the 64-bit variant; the 128-bit case follows the similar structure with adjusted block widths.

4 Evaluation

In this section, we implemented two constant-time base multipliers: `ct_poly32_mul_64_bit`, which returns padded 64-bit words, and `ct_poly32_mul_128_bit`, which extends the recursion to produce 128-bit outputs. Both variants were benchmarked on the STM32F407 Cortex-M4 under aggressive compiler optimization (`OPT=-O3`). The exact combinations of Karatsuba levels and their operation modes are summarized separately in Table 1.

4.1 Experimental Platform

All experiments were conducted on the STM32F407G-DISC1 evaluation board, equipped with an STMicroelectronics STM32F407G microcontroller. The device integrates a 32-bit ARM Cortex-M4 core with a hardware floating-point

Table 1: Karatsuba level combinations and operation modes

Combination	Operation mode
Baseline	Pure schoolbook multiplication
L3 only	Karatsuba applied at 4-word level; 2-word blocks remain schoolbook
L3+L4	Karatsuba applied at both 4-word and 2-word levels
L4 only	Karatsuba applied at 2-word level only
L2 only	Karatsuba applied at 8-word level
L2+L3	Recursive Karatsuba: $8 \rightarrow 4$ -word
L2+L3+L4	Full multi-level recursion: $8 \rightarrow 4 \rightarrow 2$
L2+L4	Top-level 8-word is schoolbook; only 2-word uses Karatsuba

unit (FPU) and operates at a maximum clock frequency of 168 MHz. It provides 1 MB of on-chip Flash memory and 192 KB of SRAM, which provide sufficient resources for cryptographic implementations. The STM32F407G-DISC1 thus represents a typical resource-constrained embedded platform for evaluating lightweight cryptographic algorithms.

4.2 Performance Evaluation

Table 2 summarizes the measured cycle counts across different Karatsuba variants in comparison with reference baselines. For reference, we include the original `poly64_mul0` [21] and constant-time `ct_poly64_mul1` [13]. Our proposed implementations (`ct_poly32_mul_64_bit` and `ct_poly32_mul_128_bit`) achieve competitive performance across multiple recursion levels (L2, L3, and L4), confirming the efficiency of the recursive design on 32-bit targets.

Table 2: Cycle counts of Karatsuba variants on Cortex-M4 (OPT=-O3)(unit : cc)

Variant	<code>poly64_mul0</code> [21]	<code>ct_poly64_mul1</code> [13]	<code>ct_poly32_mul_64_bit</code>	<code>ct_poly32_mul_128_bit</code>
Baseline	421	1,643	1,135	3,400
L3 only	421	1,641	1,171	3,392
L3+L4	421	1,641	1,207	3,407
L4 only	421	1,641	1,128	3,422
L2+L3	421	1,643	1,175	3,383
L2+L3+L4	421	1,643	1,231	3,510
L2+L4	421	1,643	1,124	3,407

Table 3 complements Table 2 by presenting the same results in relative terms. It reports the percentage differences in cycle counts with respect to the constant-time baseline (`ct_poly64_mul1` [13]) and the non-constant-time reference (`poly64_mul0` [21]).

As shown, the proposed `ct_poly32_mul_64_bit` achieves up to 31.6% improvement over the constant-time baseline (L2+L4 configuration), while deeper recursion (L2+L3+L4) slightly increases overhead due to additional subroutine calls.

Table 3: Relative performance (% difference) of Karatsuba variants on Cortex-M4

Variant	vs. <code>ct_poly64_mul1</code> [13]	vs. <code>poly64_mul0</code> [21]
Baseline	+30.92%	−169.60%
L3 only	+28.64%	−178.15%
L3+L4	+26.45%	−186.70%
L4 only	+31.26%	−167.93%
L2+L3	+28.48%	−179.10%
L2+L3+L4	+25.08%	−192.40%
L2+L4	+31.59%	−166.98%

Among our proposed designs, `ct_poly32_mul_64_bit` consistently outperformed `ct_poly32_mul_128_bit`. For the 64-bit variant, the L2+L4 configuration achieved the best result with 1,124 cycles, making it the most efficient option on Cortex-M4. In the 128-bit variant, the L2+L3 configuration was the fastest, requiring 3,383 cycles, though still about three times slower than the 64-bit design.

Across recursion levels, our implementations achieved a 25–31% improvement over the constant-time reference [13]. This gain is attributable not only to the recursive Karatsuba structure but also to reduced packing and unpacking overhead, as these operations are performed only once at the 128-bit granularity rather than repeatedly at smaller widths. Although the absolute fastest multiplier remains `poly64_mul0` [21] at 421 cycles, it is not constant-time and therefore unsuitable for secure applications.

Overall, the results demonstrate that our proposed `ct_poly32_mul_64_bit` with selective recursion (notably L2+L4) offers the best balance between efficiency and constant-time security, whereas `ct_poly32_mul_128_bit` provides scalability at the cost of higher cycle overhead.

5 Conclusion

In this paper, we presented a high-speed, constant-time implementation of radix-16 polynomial multiplication on the ARM Cortex-M4. By integrating recursive Karatsuba layers into the radix-16 zero-padding framework, we replaced the conventional schoolbook multiplier with a more efficient design while preserving constant-time execution. Our five-stage pipeline—Decomposition, Padding, Multiplication, Unpadding, and Reassembly—was carefully optimized to minimize packing overhead at the 128-bit granularity.

Experimental results on the STM32F407 Cortex-M4 demonstrated that the proposed `ct_poly32_mul_64_bit` implementation achieves up to 31% improvement over the best existing constant-time baseline, confirming the effectiveness of recursive Karatsuba on resource-constrained devices. These results highlight the practicality of our approach for accelerating polynomial arithmetic in code-based and lattice-based post-quantum cryptography on embedded microcontrollers.

Future work will primarily focus on integrating the proposed multiplier into post-quantum cryptographic schemes such as HQC and AIMer on the Cortex-M4, aiming to enhance their performance on resource-constrained embedded platforms. In addition, while our main target remains public-key primitives, the proposed multiplier could also be applied to symmetric-key settings—for example, accelerating the GHASH component of Galois/Counter Mode (GCM)—as a secondary possibility.

References

1. D. Lemire and O. Kaser, “Faster 64-bit universal hashing using carry-less multiplications,” *Journal of Cryptographic Engineering*, vol. 6, no. 3, pp. 171–185, 2016.
2. J. Lopez, V. Cadena, and M. S. Rahman, “Evaluating post-quantum cryptographic algorithms on resource-constrained devices,” *arXiv preprint arXiv:2507.08312*, 2025.
3. D. Atkins, “Requirements for post-quantum cryptography on embedded devices in the iot,” in *Third PQC Standardization Conference*, 2021.
4. M. V. Nielsen, M. R. Kjeldsen, T. Turnip, and B. Andersen, “Post-quantum digital signature algorithms on iot: Evaluating performance on lora esp32 microcontroller,” in *22nd International Conference on Security and Cryptography*, 2025.
5. D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, 3rd ed., 1997.
6. A. Karatsuba and Y. Ofman, “Multiplication of multidigit numbers on automata,” *Soviet Physics Doklady*, vol. 7, no. 7, pp. 595–596, 1963.
7. H. Seo, Z. Liu, J. Choi, and H. Kim, “Karatsuba–block-comb technique for elliptic curve cryptography over binary fields,” *Security and Communication Networks*, vol. 8, no. 17, pp. 3121–3130, 2015.
8. S. S. Erdem, “Fast software multiplication in $\mathbb{F}_2[x]$ for embedded processors,” *Turkish Journal of Electrical Engineering and Computer Sciences*, vol. 20, no. 4, pp. 593–605, 2012.
9. M. J. Kannwischer, *Polynomial multiplication for post-quantum cryptography*. PhD thesis, SI: sn, 2022.
10. M.-S. Chen and T. Chou, “Classic mceliece on the arm cortex-m4,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 125–148, 2021.
11. J. Zheng, H. Zhang, L. Tian, Z. Zhang, H. Wei, Z. Chu, Y. Yang, and Y. Zhao, “Espm-d: Efficient sparse polynomial multiplication for dilithium on arm cortex-m4 and apple m2,” *arXiv preprint arXiv:2404.12675*, 2024.
12. D. Kim, J. Choi, S. Yoon, and S. C. Seo, “Optimized implementation of hqc on cortex-m4,” *ICT Express*, 2025.
13. D. J. Bernstein. kpmc Google Group post, 2023. Code snippet: `ct_poly64_mul1` (kpmc post).

14. R. P. Brent and P. Zimmermann, *Modern Computer Arithmetic*. Cambridge University Press, 2010.
15. STMicroelectronics, “STM32F407xx datasheet.” Datasheet (device-specific; e.g. STM32F407VG), 2014. Device datasheet and electrical characteristics; see ST product page for exact part and revision.
16. STMicroelectronics, *STM32F405/407 and related devices — Reference Manual (RM0090)*, 2012. Reference manual for STM32F405xx/STM32F407xx advanced ARM-based 32-bit MCUs; check ST site for revision number.
17. M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, “pqm4: Testing and benchmarking nist pqc on arm cortex-m4,” 2019.
18. D. Owens, R. El Khatib, M. Bisheh-Niasar, R. Azarderakhsh, and M. M. Kermani, “Efficient and side-channel resistant ed25519 on arm cortex-m4,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 71, no. 6, pp. 2674–2686, 2024.
19. M. Anastasova, M. Bisheh-Niasar, H. Seo, R. Azarderakhsh, and M. M. Kermani, “Efficient and side-channel resistant design of high-security ed448 on arm cortex-m4,” in *2022 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pp. 93–96, IEEE, 2022.
20. H. Fujii and D. F. Aranha, “Curve25519 for the cortex-m4 and beyond,” in *International Conference on Cryptology and Information Security in Latin America*, pp. 109–127, Springer, 2017.
21. J. López and R. Dahab, “Fast multiplication on elliptic curves over $\text{gf}(2^m)$ without precomputation,” in *International Workshop on Cryptographic Hardware and Embedded Systems*, pp. 316–327, Springer, 1999.